

Módulo 2 — Lógica, algoritmos e fluxogramas

Jornada DEV START · Aula 2 · 14/07 (terça) · Programa START — TOTVS Paulista

Material entregue ao final da Aula 2. Guarde para consultar sempre que precisar.



Onde você está na jornada

```
1 → [2] → 3 → 4 → 5 → 6 | 7 → 8 → 9 → 10
    ▲ Fundamentos (Harbour) | Protheus (ADVPL)
    você está aqui
```

Na Aula 1 você rodou seu primeiro programa. Hoje damos um passo atrás de propósito: antes de escrever mais código, você vai aprender a **pensar como programador**. O que é um algoritmo, como desenhar a solução no papel e por que isso economiza horas na frente do teclado. É a aula que separa quem “decora comandos” de quem realmente **resolve problemas**. 🧠



Roteiro da aula (2h)

Bloco	Tempo	O quê
Retomada	~10 min	Revisão do Hello World (Aula 1) + pendências de ambiente
Conteúdo	~60 min	Como o computador “pensa”, lógica, algoritmos, fluxogramas e refinamento
Mão na massa	~40 min	Desenhar juntos o algoritmo de um problema real (pseudocódigo + fluxograma)
Q&A / networking	~10 min	Dúvidas abertas e integração da turma

Objetivos do módulo

Ao final desta aula você será capaz de: - Entender o que é um computador e como ele processa informações - Compreender o conceito de lógica e por que ela é essencial na programação - Definir e criar **algoritmos** para resolver problemas - Representar algoritmos usando **pseudocódigo** e **fluxogramas** - Aplicar a técnica de **refinamento sucessivo**

2.1 O que é um computador?

Antes de programar, é preciso entender o que estamos programando.

Um computador é uma **máquina lógica e iterativa** — ele segue instruções à risca, repete operações quantas vezes for mandado, mas **não tem iniciativa própria**. Ele faz exatamente o que você mandar, nem mais, nem menos.

O computador simplificado

Para entender melhor, imagine um computador como um **escritório simples** com:

1. **Memória** — um conjunto de “escaninhos” etiquetados onde guardamos valores. Cada escaninho é uma posição de memória identificada por um nome (que chamaremos de **variável**).
2. **CPU — Unidade de Controle** — o “chefe” que lê as instruções e decide o que fazer.
3. **CPU — ULA (Unidade Lógica e Aritmética)** — a “calculadora” que efetua operações matemáticas e comparações.
4. **Entrada (Input)** — os dados que chegam de fora: teclado, mouse, arquivo, etc.
5. **Saída (Output)** — os resultados que saem: monitor, impressora, arquivo, etc.

 Modelo entrada → processamento → saída

Exemplo de como o computador “pensa”

Imagine a instrução: “*Leia dois números e mostre a soma.*”

O computador faz assim: 1. Lê o primeiro número e guarda no escaninho **E1** 2. Lê o segundo número e guarda no escaninho **E2** 3. Soma **E1 + E2** e guarda o resultado no escaninho **E3** 4. Exibe o conteúdo de **E3**

Esse é o modelo mental que você deve carregar para sempre:

ENTRADA → PROCESSAMENTO → SAÍDA

💡 **Combinado da turma:** se a saída está errada, o computador **não errou** — ele executou com perfeição o que foi mandado. O erro está na entrada ou no algoritmo. Ou seja: o erro é nosso. Essa é a mentalidade que faz você depurar mais rápido.

2.2 O que é lógica?

“A lógica é a arte de pensar corretamente.”

No dia a dia você usa lógica sem perceber. Quando segue uma receita de bolo, quando escolhe a rota mais rápida para o trabalho, quando calcula o troco — está usando lógica.

Em programação, lógica é a **capacidade de organizar o raciocínio em passos claros e sem ambiguidade**, de forma que um computador possa executá-los.

Por que a lógica vem antes da linguagem?

Um computador é ótimo em executar instruções, mas é completamente “literal” — ele não interpreta, não adivinha, não usa senso comum. Se você mandar fazer algo errado, ele fará errado com total eficiência.

Por isso a lógica vem **antes** da programação. A linguagem (Harbour, ADVPL, qualquer outra) é apenas o **veículo** para expressar o seu raciocínio. Primeiro você pensa; depois traduz para código.

Exemplo de raciocínio lógico (dedução)

- “Todo ser humano é mortal.”
- “Sócrates é ser humano.”
- Logo: **“Sócrates é mortal.”**

Esse tipo de raciocínio — de **premissas** para **conclusão** — é a base do que fazemos ao programar: dadas as condições, qual é o resultado?

```
Partimos de → condições (premissas)
Aplicamos   → regras (algoritmo)
Chegamos a  → resultados (saída)
```

2.3 Algoritmos

O que é um algoritmo?

A palavra “algoritmo” vem do nome do matemático persa **Al-Khwarismi** (século IX), autor de tratados sobre aritmética. O conceito é muito **anterior** aos computadores.

Algoritmo é uma sequência **finita, ordenada e sem ambiguidade** de passos que levam à solução de um problema.

Repare nos três requisitos: - **Finita:** precisa terminar. Um algoritmo que nunca termina é inútil. - **Ordenada:** a ordem importa. Executar o passo 3 antes do 2 pode gerar resultado errado. - **Sem ambiguidade:** cada passo deve ser claro e ter apenas uma interpretação.

Algoritmos no dia a dia

O exemplo clássico é uma **receita culinária**: 1. Pegue 2 ovos 2. Misture com 1 xícara de farinha 3. Adicione meia xícara de leite 4. Mexa até a massa ficar homogênea 5. Despeje numa forma untada 6. Asse a 180°C por 30 minutos

Os passos são finitos, ordenados e sem ambiguidade. Outros exemplos do cotidiano: instruções de montagem de um móvel, o trajeto que o GPS dita, a bula de um remédio.

Características de um bom algoritmo

Característica	Descrição
Clareza	Cada instrução define de forma inequívoca o que deve ser feito
Finitude	Sempre termina após um número finito de passos
Sequência	Passos em ordem cronológica correta
Eficiência	Consome o mínimo de tempo e recursos possível

“O melhor algoritmo é aquele que, construído com elegância e simplicidade, atinge o objetivo de maneira eficiente.”

2.4 Como representar um algoritmo

Existem três formas principais de representar um algoritmo **antes** de codificá-lo.

1. Linguagem natural

Escrever o algoritmo em português comum. Simples, mas pode ser **ambíguo**.

Exemplo: “Leia dois números, some-os e exiba o resultado.”

2. Pseudocódigo

Uma linguagem intermediária, parecida com código real, mas em português. Mais **precisa** que a linguagem natural — e muito próxima do que você vai digitar depois.

```
Início
  Leia A
  Leia B
  Resultado ← A + B
  Escreva Resultado
Fim
```

3. Fluxograma

Representação **gráfica** usando símbolos padronizados. Ótimo para enxergar o fluxo de decisões.

Símbolos principais:

Símbolo	Formato	Significado
Terminal	Oval / arredondado	Início ou Fim
Processo	Retângulo	Operação, cálculo, atribuição
Decisão	Losango	Condição (pergunta com Sim/Não)
Entrada/Saída	Paralelogramo	Leitura ou escrita de dados
Conector	Círculo	Continua em outro ponto

 Símbolos padronizados de fluxograma

Exemplo — calcular a média de dois números:

```
[Início]
  ↓
[Leia N1, N2]
  ↓
[Média ← (N1 + N2) / 2]
  ↓
[Escreva Média]
  ↓
[Fim]
```

 Fluxograma do cálculo da média

2.5 O ciclo de desenvolvimento

Todo bom programador segue um ciclo **antes** de sentar na frente do computador:

1. DOMÍNIO DO PROBLEMA
↓
Leia o enunciado. Entenda completamente o que é pedido.
Anote os dados de entrada e a saída esperada.
2. DOMÍNIO DA SOLUÇÃO
↓
Pense na solução. Use lápis e papel.
Crie o fluxograma ou o pseudocódigo.
Teste mentalmente com dados de exemplo.
3. IMPLEMENTAÇÃO
↓
Escreva o código na linguagem escolhida.
Compile e teste.
Corrija os erros (debug).

Lápis e papel antes do teclado. Desenvolvedores experientes nunca pulam essa etapa.

Por que isso importa?

Corrigir um erro de lógica **no papel** é muito mais rápido do que corrigir no código. Um bug descoberto na fase de implementação pode custar horas; o mesmo bug descoberto no papel custa segundos.

2.6 Refinamento sucessivo

Quando um problema é grande ou complexo, é difícil criar a solução de uma vez só. A técnica do **refinamento sucessivo** resolve isso: você enxerga o problema em **camadas**, começando pela visão mais geral e adicionando detalhes a cada nível.

Exemplo: calcular a média escolar de um aluno

Nível 1 — visão geral:

```
Início
  Obter dados do aluno
  Calcular média
  Mostrar resultado
Fim
```

Nível 2 — detalhando cada parte:

```
Obter dados do aluno:
  Ler nome do aluno
  Ler nota do 1º bimestre
  Ler nota do 2º bimestre
  Ler nota do 3º bimestre
  Ler nota do 4º bimestre

Calcular média:
  Média ← (N1 + N2 + N3 + N4) / 4

Mostrar resultado:
  Se Média >= 7
    Escrever "Aprovado. Média: " + Média
  Senão
    Escrever "Reprovado. Média: " + Média
```

Por que usar refinamento sucessivo?

- Divide o problema em partes menores (mais fáceis de resolver)
- Cada parte pode ser desenvolvida e testada separadamente
- Resulta naturalmente em um programa **bem modular**
- Facilita a manutenção futura

Este é o fundamento da **programação modular**, que você estudará no Módulo 6.

2.7 Linguagens de programação

O que é uma linguagem de programação?

É um conjunto de instruções e regras (sintaxe e semântica) que permitem dar ordens ao computador. Cada linguagem tem suas próprias palavras-chave, regras e propósitos.

O processo de tradução

O código que você escreve (**código-fonte**) precisa ser traduzido para algo que o processador entenda (**código de máquina**). Isso é feito por:

Compilador - Lê todo o código-fonte de uma vez - Gera um arquivo executável (`.exe` , `.out`)

- O arquivo roda **sem** precisar do compilador junto - Exemplos: C, C++, Pascal,

Harbour/ADVPL

Interpretador - Traduz e executa o código linha por linha, em tempo real - Precisa estar instalado para o programa rodar - Exemplos: Python, JavaScript, Ruby

Harbour/ADVPL é uma linguagem compilada — foi exatamente o que você fez na Aula 1 com o `hbm2` : transformou o `.prg` (fonte) em `.exe` (executável).

Níveis de linguagem

```
Código de máquina (binário 01010101)    ← mais difícil para humanos
      ↑
Assembly (instruções simbólicas)
      ↑
Linguagens de alto nível                  ← mais fácil para humanos
(C, Pascal, Python, Harbour, ADVPL...)
```

Harbour e ADVPL são de **alto nível** — mais próximas da linguagem humana, o que as torna mais produtivas.

2.8 Boas práticas de código

Antes de escrever mais programas, conheça as regras que todo bom programador segue. Elas valem para o resto do curso.

1. Uma instrução por linha

```
// Errado (difícil de ler)
A := 1 ; B := 2 ; C := A + B

// Correto
A := 1
B := 2
C := A + B
```

2. Indentação (recuo)

Blocos de código dentro de estruturas devem ser recuados (geralmente 3 ou 4 espaços):

```
IF nIdade >= 18
    QOut("Maior de idade") // recuado – pertence ao IF
ENDIF
```

3. Comentários com propósito

```
// Bom: explica o POR QUÊ
// Aplicamos desconto de 15% para clientes com mais de 5 anos
nDesconto := nValor * 0.15

// Ruim: apenas repete o que o código já diz
// Multiplica valor por 0.15
nDesconto := nValor * 0.15
```

4. Nomes descritivos

```
// Ruim
LOCAL x, y, z

// Bom
LOCAL nSalario, nDesconto, nSalarioLiquido
```

Mão na massa (em aula)

Vamos resolver **juntos**, no papel/quadro, sem tocar no teclado ainda:

Problema: “Uma loja dá desconto de 10% para compras acima de R\$ 100. Leia o valor da compra e mostre o valor final a pagar.”

Faremos em três passos, aplicando tudo da aula:

1. **Domínio do problema** — qual é a entrada? qual é a saída? qual é a regra?
2. **Pseudocódigo** — escrever a solução em português estruturado:

```
Início
  Leia Valor
  Se Valor > 100
    Desconto ← Valor * 0.10
  Senão
    Desconto ← 0
  Total ← Valor - Desconto
  Escreva Total
Fim
```

3. **Fluxograma** — desenhar o mesmo raciocínio com os símbolos (terminal, E/S, decisão em losango, processo).

No fim, testamos mentalmente com dois exemplos: uma compra de R\$ 80 (sem desconto) e uma de R\$ 250 (com desconto). 🎯

Referência rápida do Módulo 2

MODELO MENTAL

ENTRADA → PROCESSAMENTO → SAÍDA

ALGORITMO = passos finitos, ordenados e sem ambiguidade

CICLO: entender o problema → resolver no papel → implementar/testar

PSEUDOCÓDIGO

Início / Fim

Leia X (entrada)

Escreva X (saída)

X ← valor (atribuição, o "recebe")

Se ... Senão (decisão)

SÍMBOLOS DE FLUXOGRAMA

Oval → Início / Fim

Retângulo → Processo / cálculo

Losango → Decisão (Sim / Não)

Paralelogramo → Entrada / Saída

Círculo → Conector

TRADUÇÃO

Compilador → gera .exe (Harbour/ADVPL, C, Pascal)

Interpretador → roda linha a linha (Python, JavaScript)



Exercícios

Os exercícios desta aula estão em `exercicios.md` (nesta mesma pasta). **Prazo sugerido:** até a Aula 3 (15/07).



Para a próxima aula

Módulo 3 — Variáveis, tipos e operadores. Agora que você sabe **pensar** a solução, vamos começar a escrever código de verdade: como o computador guarda informações nos “escaninhos”, quais **tipos de dados** existem em Harbour/ADVPL e como manipulá-los com **operadores**.

👤 Antes da Aula 3: tente resolver ao menos os exercícios de pseudocódigo. Travou em algum fluxograma? Poste uma foto no Discord [#duvidas-parte1-harbour](#) — a gente destrava junto.